

Reviewer Pack — Minimal Local System Rank Correlation with Resolution Proof ...

Entry d5964e3317ae · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 01:21:50 UTC

Minimal Local System Rank Correlation with Resolution Proof Width via Symplectic Geometry

Entry ID: d5964e3317ae **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 01:21:50 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given boolean formula φ with n variables, the resolution proof width $w(\varphi)$ is linearly correlated with the minimal local system rank ($\text{mls}(\varphi)$) of the symplectic embedding of its associated tautological variety $V(\varphi)$, such that $\text{mls}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Symplectic geometry has been historically underutilized in complexity theory despite offering powerful tools for studying geometric structures. The minimal local system rank, a measure of the complexity of symplectic embeddings, could potentially capture intricate properties of resolution proof trees that are not easily captured by other means.

Taxonomy category: symplectic_geometry_resolution_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8bd2c99f678b45ae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all 30 random seeds, the correlation coefficient between minimal local system rank ($\text{mls}(\varphi)$) and resolution proof width ($w(\varphi)$) is greater than or equal to 0.8 AND the mean absolute difference between $\text{mls}(\varphi)$ and $w(\varphi)$ across all seeds is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        if n == 1:
            return "x"
        else:
            p = random.choice(["and", "or"])
            left = generate_formula(random.randint(1, n-1))
            right = generate_formula(random.randint(1, n-1))
            return f"({left} {p} {right})"

    def resolution_width(formula):
        stack = []
        for token in formula.split():
            if token == "and":
                stack.append(stack.pop() + " and " + stack.pop())
            elif token == "or":
                stack.append(stack.pop() + " or " + stack.pop())
            else:
                stack.append(token)
        return len(stack[0].split()) - 1

    def tautological_variety(formula):
        # Placeholder for actual computation
        return formula

    def minimal_local_system_rank(variety):
        # Placeholder for actual computation
        return len(variety.split())

    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_formula(n)
```

```

variety = tautological_variety(formula)
mls_phi = minimal_local_system_rank(variety)
w_phi = resolution_width(formula)

return {
    "metric_name": "mls_w_correlation",
    "metric_value": mls_phi / (w_phi + 1), # Avoid division by zero
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False, # Mapping undefined for this conjecture
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d53c93bf.py", line 73, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d53c93bf.py", line 53, in run_trial

```

    w_phi = resolution_width(formula)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d53c93bf.py", line 36, in resolution_width

```

    stack.append(stack.pop() + " or " + stack.pop())
    ~~~~~

```

IndexError: pop from empty list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15958
2	propose	ollama_remote	glm4:latest	0	0	24481
3	preregistration	ollama_remote	glm4:latest	0	0	9621
4	novelty	ollama_remote	glm4:latest	0	0	29245
5	novelty	ollama_remote	glm4:latest	0	0	14566
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17138
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21986
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15256
10	judge	ollama_remote	glm4:latest	0	0	8809

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167672 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d5964e3317ae.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d5964e3317ae.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d5964e3317ae.tar.gz` (if generated)